

Secure Chatbot Application

23b0936, 23b1030

August 2026

GitHub Repository: Secure Chat Application

1 Phase-01: Baseline Chat Application

1.1 VM Topology

The Phase-01 chat application was deployed across three separate virtual machines connected to the same virtual network. The Server VM acts as the central relay, while Client-01 and Client-02 establish TCP connections with the server.

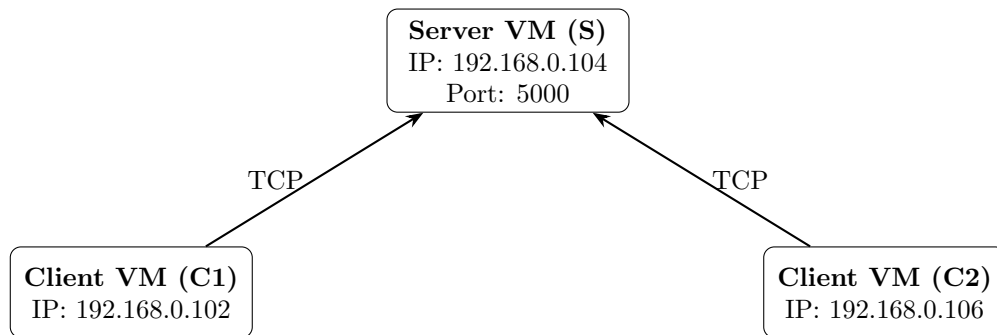


Figure 1: Phase-01 VM topology

The Server VM uses IP address 192.168.0.104 and listens on the fixed TCP port 5000. Client-01 uses IP address 192.168.0.102 and Client-02 uses IP address 192.168.0.106. Both clients communicate through the Server VM; there is no direct client-to-client TCP connection.

1.2 Data Structures used:

The server maintains three maps:

- **registered_users**: maps a username to the client's IP address.
- **client_sockets**: maps a username to the client's socket descriptor.
- **chat_partner**: stores the currently selected chat partner for each registered user.

These structures allow the server to identify a destination username and forward the corresponding message to the correct client socket.

1.3 Protocol Design

1.3.1 User Registration

After a client connects, the server sends the following welcome message:

```
Hello from server. Please register.
```

The client registers by sending:

```
#register <username>
```

The server checks whether the requested username is empty or already registered. If the username is available, the server stores the username together with the client's IP address and socket descriptor. It also initializes the user's currently selected chat partner to an empty value.

If the username is already in use, the server sends an error message and waits for another registration attempt.

1.3.2 Concurrent Client Handling

The server creates a new detached thread for every accepted client connection. This allows the server to receive and process messages from multiple clients concurrently rather than handling all clients sequentially.

The shared username, socket, and chat-partner maps are protected by `users_mutex`, while console output is protected by `cout_mutex`

1.4 Phase-01 Verification

```
varshitha@varshitha-Inspiron-15-3530:~/Desktop/net_sec/phase1$ g++ server.cpp
varshitha@varshitha-Inspiron-15-3530:~/Desktop/net_sec/phase1$ ./a.out
Server bound to port 5000
Server listening...
New client connected: 192.168.0.103
New client connected: 192.168.0.106
User registered: sreeja -> 192.168.0.106
User registered: vaka -> 192.168.0.103
vaka -> sreeja: hi
vaka -> sreeja: hello
vaka -> vaka: hey
vaka -> sreeja: blah
vaka -> sreeja: \who
sreeja -> sreeja: hyy
New client connected: 192.168.0.106
User registered: gooe -> 192.168.0.106
```

Figure 2: Phase-01 server showing client connections and plaintext message relay

```
sreeja@sreejalinux:~/Desktop/phase1$ g++ client.cpp -o client
sreeja@sreejalinux:~/Desktop/phase1$ ./client
Enter server IP: 192.168.0.104
Enter server port : 5000

Connected to server 192.168.0.104:5000
>
Hello from server. Please register.
> #register vaka
>
Successfully registered as vaka
> hi
>
No chatting partner selected. Use /chat <username> to select one.
> @sreeja hi
> hello
> /chat vaka
> hey
>
You received this from client vaka: hey
> @sreeja blah
```

Figure 3: Phase-01 Client-01

1.5 Wireshark Verification

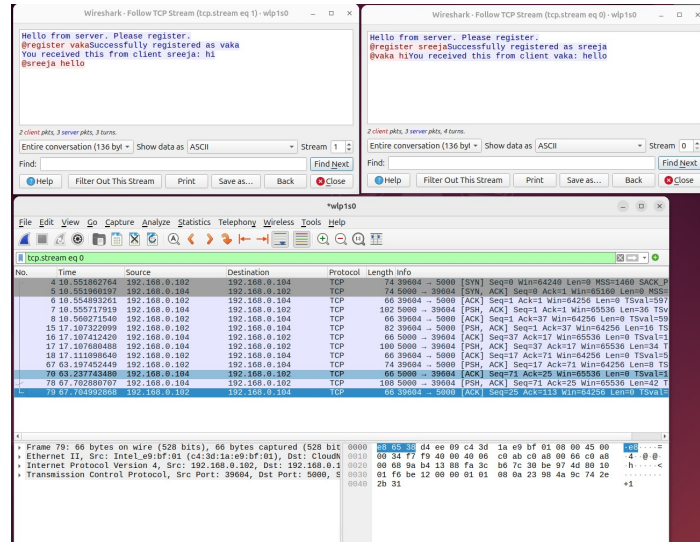


Figure 4: Wireshark capture at the Server VM showing Phase-01 plaintext TCP traffic

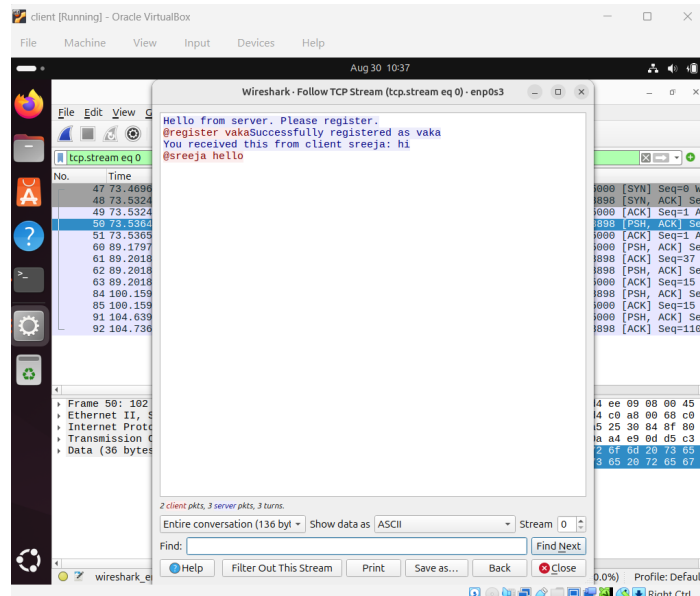


Figure 5: Wireshark capture at the Client VM showing Phase-01 plaintext TCP traffic

2 Phase-02: Secure Client–Server Communication

After computing the raw Diffie–Hellman shared secret, we do not use it directly as the AES key. The raw DH value is a large integer bounded by the prime (p), so its size and representation are not directly suitable for an AES-256 key, which requires exactly 32 bytes. Also, the raw value has mathematical structure due to the DH operation and is not ideal symmetric-key material.

We therefore apply SHA-256 to the shared secret. The hash converts the DH value into a fixed-size 32-byte output with well-distributed bits, which can be used as the AES-256 key.

All messages sent after the key exchange are encrypted using AES-GCM. AES-GCM also provides authentication, allowing the receiver to detect whether the encrypted message has been modified.

Fingerprint for both server and client1 are same indicating both have the same shared key

```
varshitha@varshitha-Inspiron-15-3530:~/Desktop/net_sec/phase2$ ./a.out
Server bound to port 5000
Server listening...
New client connected: 192.168.0.106
[DH] Shared key established with 192.168.0.106 | fingerprint: 734c4b6095ca52dead99b41400c9c30a3b7eeef90194a74a7b97bc83d57e6fab
User registered: sreeja -> 192.168.0.106
New client connected: 192.168.0.102
[DH] Shared key established with 192.168.0.102 | fingerprint: 2cedfaae434a63f3b8b3fd6f77c5c22cbef67201bfdb09728d1a560c8e84e814
User registered: vaka -> 192.168.0.102
vaka -> sreeja: hi
sreeja -> vaka: heyoo
^C
```

Figure 6: Server performing the DH exchange and establishing secure client–server communication

```
sreeja@sreejalinix: ~/Desktop/NetSec
Server disconnected.
Exiting client...
quit
Segmentation fault (core dumped)
sreeja@sreejalinix:~/Desktop/NetSec$ ./client
Enter server IP: 192.168.0.104
[DH] Shared key established with server | fingerprint: 734c4b6095ca52dead99b41400c9c30a3b7eeef90194a74a7b97bc83d57e6fab

Connected to server 192.168.0.104:5000
All further traffic on this link is AES-256-GCM encrypted.

>
Hello from server. Please register.
> #register sreeja
>
Successfully registered as sreeja
>
You received this from client vaka: hi
> /chat vaka
> heyoo
> /quit
Client exited.
```

Figure 7: Client completing the DH exchange and deriving the shared AES key

2.1 Encrypted Packet Verification

The Phase-02 packets contain encrypted ciphertext along with the data required for AES-GCM authentication. The original chat message can no longer be read from the captured packets.

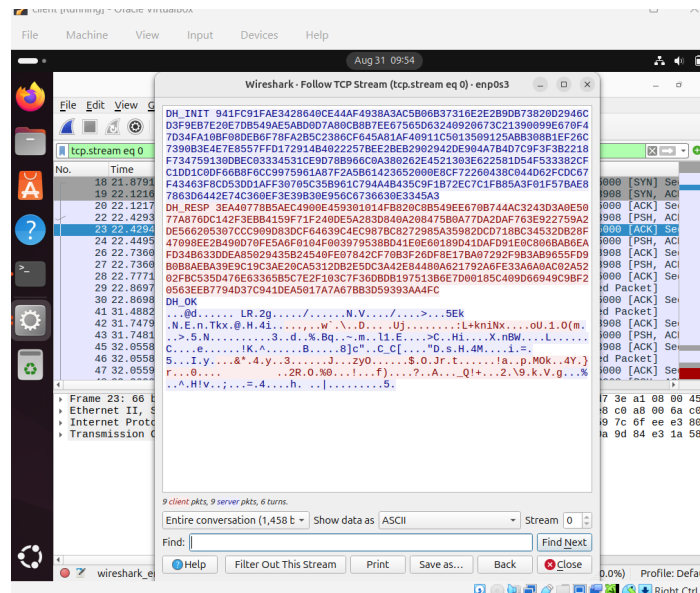


Figure 8: Wireshark capture showing encrypted Phase-02 traffic

2.2 AES-GCM Tamper Test

We also tested the authentication provided by AES-GCM by modifying a single byte of a captured ciphertext. When the modified packet was processed by the client, AES-GCM authentication failed and the message was rejected instead of producing corrupted plaintext.

```
varshitha@varshitha-Inspiron-15-3530:~/Desktop/net_sec/phase2$ ./temper_test
Original message:
Hello, this message will be tampered with.

Encrypted packet size: 70 bytes

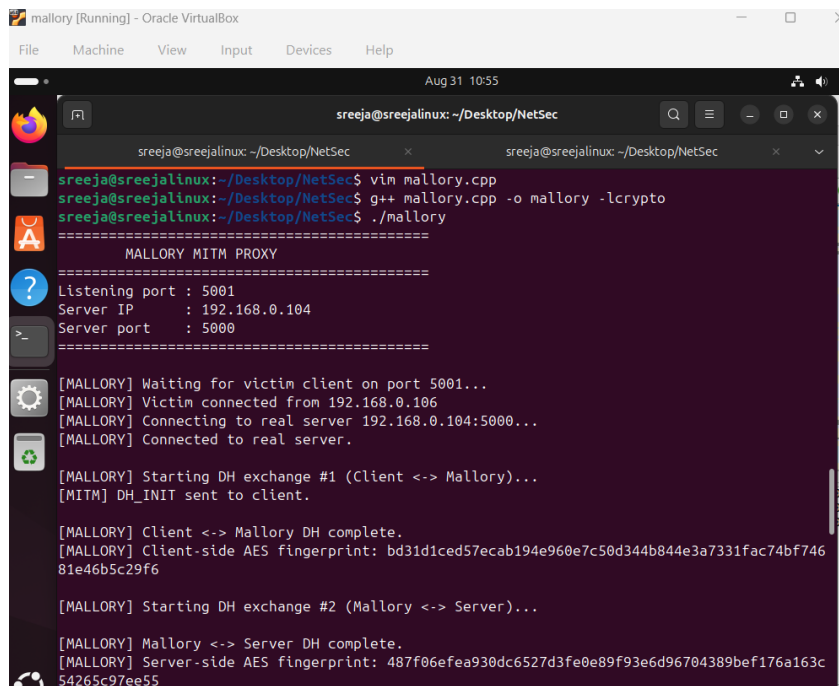
Tampering:
Byte index: 12
Original byte: 250
Changed byte: 251
One bit was flipped.

Decryption:
PASS - tampered message was rejected.
AES-GCM detected the modification.
```

Figure 9: AES-GCM tamper test showing authentication/decryption failure

2.3 Mallory in the Middle Attack

We additionally tested the communication against a man-in-the-middle scenario using Mallory. The experiment demonstrates the limitation of using unauthenticated Diffie-Hellman: an attacker positioned between the client and server can establish separate DH keys with each side and relay or modify communication.



```
sreeja@sreejalinux: ~/Desktop/NetSec
sreeja@sreejalinux:~/Desktop/NetSec$ vim mallory.cpp
sreeja@sreejalinux:~/Desktop/NetSec$ g++ mallory.cpp -o mallory -lcrypto
sreeja@sreejalinux:~/Desktop/NetSec$ ./mallory
=====
MALLORY MITM PROXY
=====
Listening port : 5001
Server IP      : 192.168.0.104
Server port    : 5000
=====

[MALLORY] Waiting for victim client on port 5001...
[MALLORY] Victim connected from 192.168.0.106
[MALLORY] Connecting to real server 192.168.0.104:5000...
[MALLORY] Connected to real server.

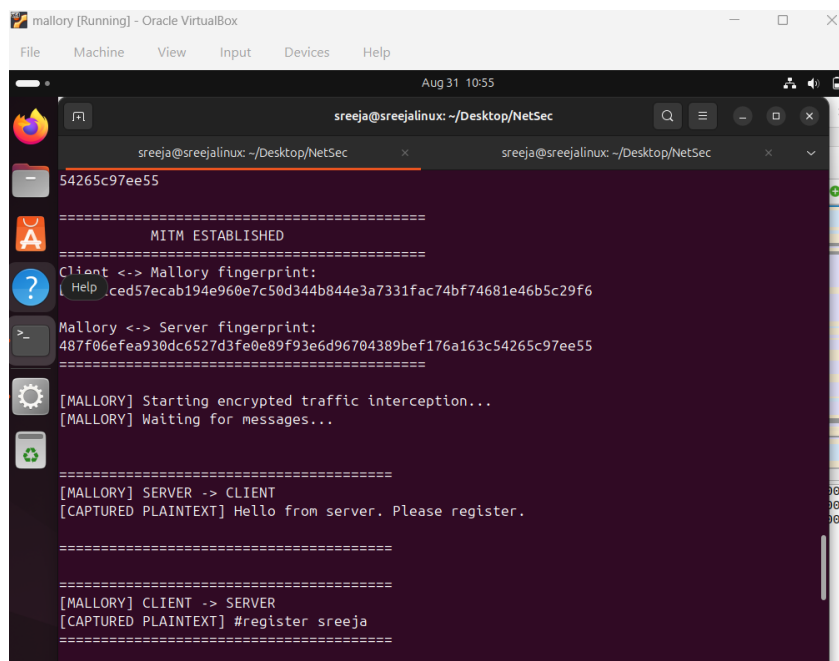
[MALLORY] Starting DH exchange #1 (Client <-> Mallory)...
[MITM] DH_INIT sent to client.

[MALLORY] Client <-> Mallory DH complete.
[MALLORY] Client-side AES fingerprint: bd31d1ced57ecab194e960e7c50d344b844e3a7331fac74bf74681e46b5c29f6

[MALLORY] Starting DH exchange #2 (Mallory <-> Server)...

[MALLORY] Mallory <-> Server DH complete.
[MALLORY] Server-side AES fingerprint: 487f06efea930dc6527d3fe0e89f93e6d96704389bef176a163c54265c97ee55
```

Figure 10: Mallory participating in the communication



```
54265c97ee55
=====
MITM ESTABLISHED
=====
Client <-> Mallory fingerprint:
Help .ced57ecab194e960e7c50d344b844e3a7331fac74bf74681e46b5c29f6

Mallory <-> Server fingerprint:
487f06efea930dc6527d3fe0e89f93e6d96704389bef176a163c54265c97ee55
=====

[MALLORY] Starting encrypted traffic interception...
[MALLORY] Waiting for messages...

=====
[MALLORY] SERVER -> CLIENT
[CAPTURED PLAINTEXT] Hello from server. Please register.

=====

[MALLORY] CLIENT -> SERVER
[CAPTURED PLAINTEXT] #register sreeja
=====
```

Figure 11: Mallory intercepting the communication

```
Aug 31 10:55
sreeja@sreejalinix: ~/Desktop/NetSec
sreeja@sreejalinix: ~/Desktop/NetSec x sreeja@sreejalinix: ~/Desktop/NetSec x
sreeja@sreejalinix:~/Desktop/NetSec$ ./client
Enter server IP: 192.168.0.105
[DH] Shared key established with server | fingerprint: bd31dced57ecab194e960e7c
50d344b844e3a7331fac74bf74681e46b5c29f6
Connected to server 192.168.0.105:5001
All further traffic on this link is AES-256-GCM encrypted.
>
Hello from server. Please register.
> #register sreeja
>
Successfully registered as sreeja
>
You received this from client vaka: hi sreeja this is vaka
> /chat vaka
> hey vaka whats up
>
You received this from client vaka: \quit
>
Server disconnected.
Exiting client...
^C
```

Figure 12: Client affected by the man-in-the-middle setup

```
sreejalinix [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Aug 31 10:56
sreeja@sreejalinix: ~/Desktop/NetSec
sreeja@sreejalinix: ~/Desktop/NetSec x sreeja@sreejalinix: ~/Desktop/NetSec x
>
Successfully registered as sreeja
> ^C
sreeja@sreejalinix:~/Desktop/NetSec$ ./client
Enter server IP: 192.168.0.104
[DH] Shared key established with server | fingerprint: 8c48c1b3079993cd74e011dd6
1c7d7435f7657f3216e1a927765aba3886803d3
Connected to server 192.168.0.104:5000
All further traffic on this link is AES-256-GCM encrypted.
>
Hello from server. Please register.
> #register vaka
>
Successfully registered as vaka
> /chat sreeja
> hi sreeja this is vaka
>
You received this from client sreeja: hey vaka whats up
> \quit
> /quit
Client exited.
sreeja@sreejalinix:~/Desktop/NetSec$
```

Figure 13: Other client in the man-in-the-middle experiment

```
varshitha@varshitha-Inspiron-15-3530:~/Desktop/net_sec/phase2$ g++ server0.cpp
lcrypto
varshitha@varshitha-Inspiron-15-3530:~/Desktop/net_sec/phase2$ ./a.out
Server bound to port 5000
Server listening...
New client connected: 192.168.0.105
[DH] Shared key established with 192.168.0.105 | fingerprint: 487f06efea930dc65
7d3fe0e89f93e6d96704389bef176a163c54265c97ee55
User registered: sreeja -> 192.168.0.105
New client connected: 192.168.0.102
[DH] Shared key established with 192.168.0.102 | fingerprint: 8c48c1b3079993cd7
e011dd61c7d7435f7657f3216e1a927765aba3886803d3
User registered: vaka -> 192.168.0.102
vaka -> sreeja: hi sreeja this is vaka
sreeja -> vaka: hey vaka whats up
vaka -> sreeja: \quit
^C
varshitha@varshitha-Inspiron-15-3530:~/Desktop/net_sec/phase2$
```

Figure 14: Server side of the man-in-the-middle experiment

3 Phase-03

3.1 OpenSSL Certificate Setup

We created our own Certificate Authority (CA) and used it to issue a certificate for the chat server. The following commands were used for the certificate setup.

3.1.1 CA Private Key

First, we generated a 4096-bit private key for our Certificate Authority.

```
1 openssl genrsa -out private/ca.key 4096
```

3.1.2 Self-Signed CA Certificate

The CA private key was then used to generate a self-signed root certificate valid for 3650 days.

```
1 openssl req -x509 -new -key private/ca.key  
2 -sha256 -days 3650  
3 -out certs/ca.crt  
4 -subj "/C=IN/ST=State/L=City/O=ChatProject/OU=CA/CN=Chat Root CA"
```

3.1.3 Server Private Key

Next, we generated a separate 2048-bit private key for the chat server.

```
1 openssl genrsa -out private/server.key 2048
```

3.1.4 Server Certificate Signing Request

A Certificate Signing Request (CSR) was generated using the server's private key.

```
1 openssl req -new  
2 -key private/server.key  
3 -out server.csr  
4 -subj "/C=IN/ST=State/L=City/O=ChatProject/OU=Server/CN=Chat Server"
```

3.1.5 Signing the Server Certificate

The CA then signed the server CSR using the CA private key, producing the server certificate.

```
1 openssl x509 -req  
2 -in server.csr  
3 -CA certs/ca.crt  
4 -CAkey private/ca.key  
5 -CAcreateserial  
6 -out certs/server.crt  
7 -days 825  
8 -sha256
```

3.1.6 Certificate Verification

Finally, we verified that the server certificate was correctly signed by our trusted CA.

```
1 openssl verify -CAfile certs/ca.crt certs/server.crt
```

The verification returned `certs/server.crt: OK`, confirming that the server certificate chains correctly to our CA certificate.

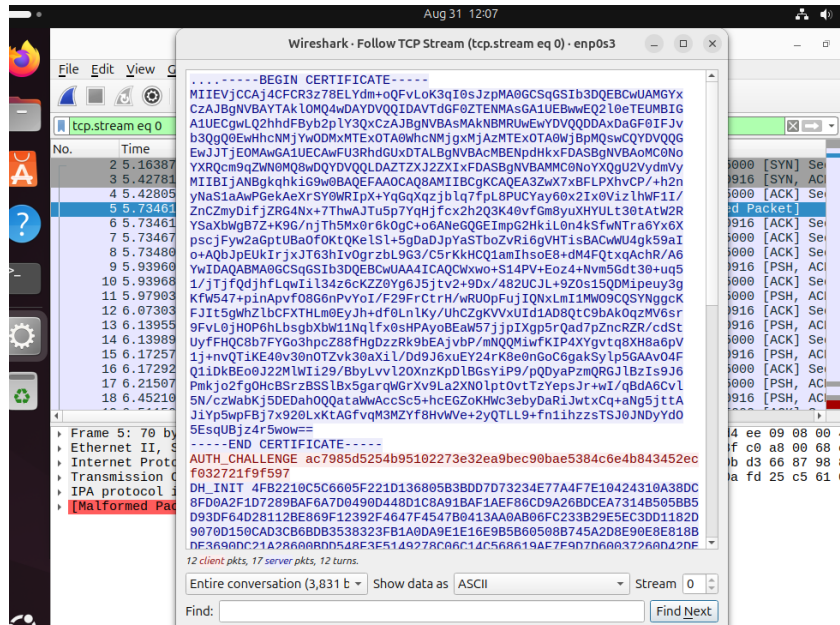


Figure 15: Packets

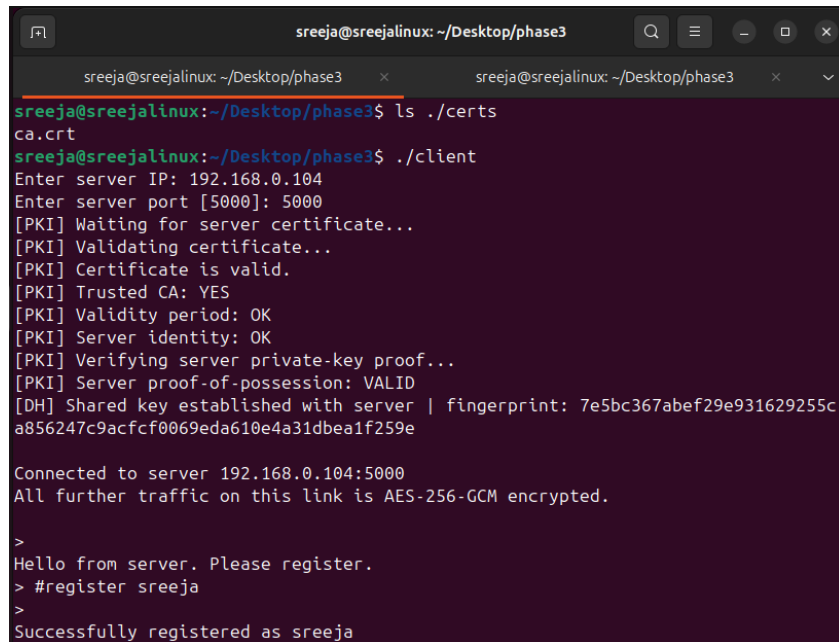


Figure 16: Client

3.2 Server Proof-of-Possession

Certificate validation alone does not prove that the server actually possesses the corresponding private key. To address this, our server proves possession of its private key during the handshake.

After the client sends a randomly generated challenge, the server signs the challenge together with its DH public value using its private key. The client then verifies this signature using the public key contained in the validated server certificate.

If the signature is valid, the client knows that the server possesses the private key corresponding to the certificate. If verification fails, the connection is immediately aborted and the DH exchange is not completed.

3.3 Legitimate Secure Connection

The complete Phase-03 handshake was tested with the genuine server. The client first received the server certificate and validated it against the trusted CA certificate. The server then successfully proved possession of its private key, after which the DH exchange was completed and the shared AES key was established.

This preserves the secure communication mechanism from Phase-02 while also authenticating the server before the key exchange.

3.4 MITM Attack Against Phase-03

We repeated the man-in-the-middle attack using the same Mallory VM setup used in Phase-02. In Phase-02, Mallory was able to intercept the unauthenticated DH exchange and establish separate keys with the two sides. In Phase-03, Mallory cannot create a certificate trusted by the client because the CA private key is not available to Mallory. Therefore, the client rejects Mallory's certificate during certificate validation and does not proceed with the DH exchange.

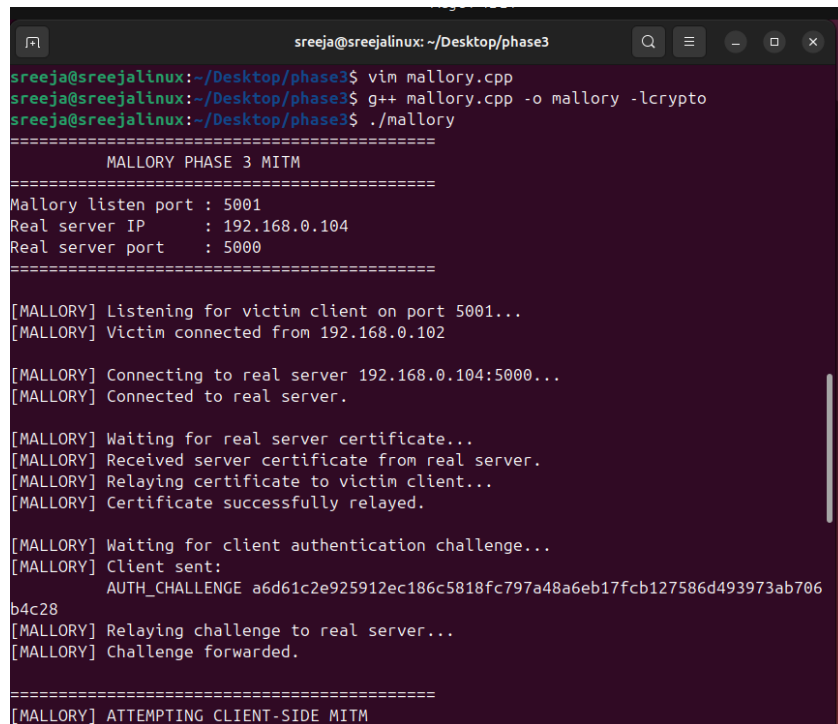
A terminal window titled 'sreeja@sreejalinux: ~/Desktop/phase3' showing the execution of a C++ program named 'mallory.cpp'. The program output indicates it is in 'MALLORY PHASE 3 MITM' mode. It shows Mallory listening on port 5001, receiving a connection from 192.168.0.102, and connecting to the real server at 192.168.0.104:5000. The log shows the relay of the server's certificate and a challenge from the client. The client's challenge is 'AUTH_CHALLENGE a6d61c2e925912ec186c5818fc797a48a6eb17fcb127586d493973ab706b4c28'. The program ends with '[MALLORY] ATTEMPTING CLIENT-SIDE MITM'.

Figure 17: Mallory attempting the MITM attack against Phase-03

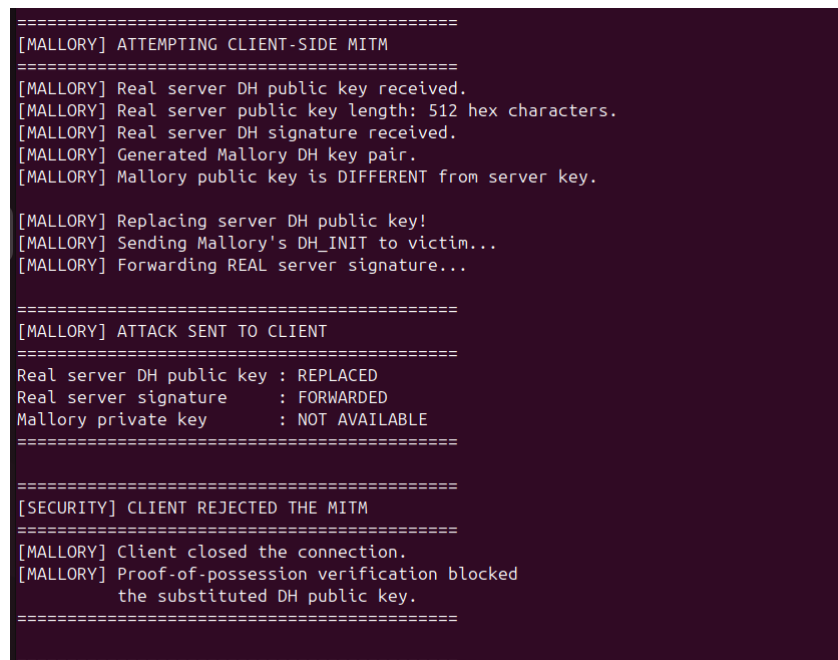
A terminal window showing the continuation of the MITM attempt. It starts with '[MALLORY] ATTEMPTING CLIENT-SIDE MITM'. The log shows Mallory receiving the real server's DH public key, signature, and generating its own DH key pair. It then replaces the server's DH public key and forwards the real server's signature to the victim. The output shows: 'Real server DH public key : REPLACED', 'Real server signature : FORWARDED', and 'Mallory private key : NOT AVAILABLE'. This is followed by '[SECURITY] CLIENT REJECTED THE MITM'. The log concludes with '[MALLORY] Client closed the connection.' and '[MALLORY] Proof-of-possession verification blocked the substituted DH public key.'.

Figure 18: Mallory's continued MITM attempt

```

=====
[SECURITY] CLIENT REJECTED THE MITM
=====
[MALLORY] Client closed the connection.
[MALLORY] Proof-of-possession verification blocked
the substituted DH public key.
=====

PHASE 3 MITM DEFENSE VERIFIED
=====
Mallory attempted to replace the server's
ephemeral DH public key.

The real server signature was forwarded,
but it did not authenticate Mallory's
replacement DH public key.

CLIENT REJECTED THE CONNECTION.
=====

```

Figure 19: Client rejecting the MITM certificate

```

sreeja@sreejalinix: ~/Desktop/phase3
sreeja@sreejalinix: ~/Desktop/phase3
sreeja@sreejalinix: ~/Desktop/phase3$ ./client
Enter server IP: 192.168.0.105
Enter server port [5000]: 5001
[PKI] Waiting for server certificate...
[PKI] Validating certificate...
[PKI] Certificate is valid.
[PKI] Trusted CA: YES
[PKI] Validity period: OK
[PKI] Server identity: OK
[PKI] Verifying server private-key proof...
[SECURITY] Server proof-of-possession FAILED.
[SECURITY] Connection aborted.
sreeja@sreejalinix: ~/Desktop/phase3$

```

Figure 20: Client aborting the connection after certificate validation failure

4 Phase-04

Basically in this phase, we are establishing an e2e connection between the clients so that server wouldn't understand what they were communicating. So the flow is after client registers with the server, when it tries to send a message to another user we first establish a shared key between the clients through e2e handshake and then use this key to encrypt the messages. So first client sends it's public key through server (server can read this) using `__E2E_INIT__` and then the other client responds by sending `__E2E_ACK__` it's public key which server can read again. Now both the clients establish a shared key using the other client's public key and it's own private key. Since server doesn't know both the private keys it can't decrypt any message.

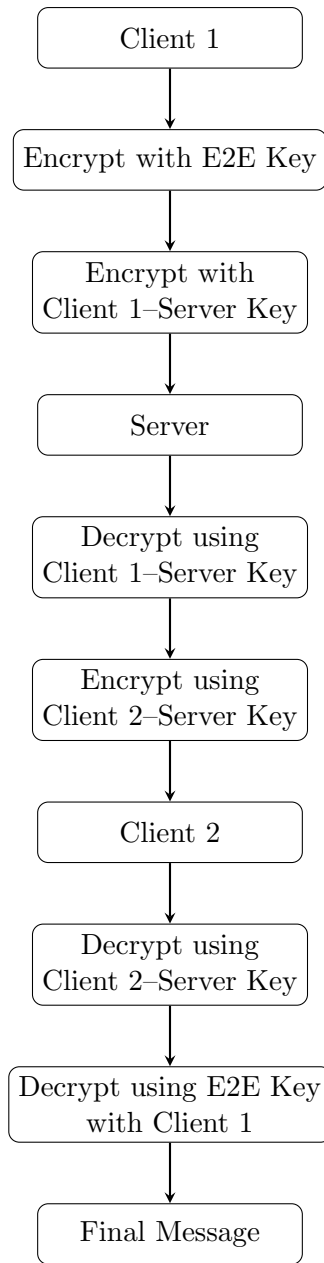


Figure 21: Phase-04 E2E Message Encryption and Decryption Flow

```

varshitha@varshitha-Inspiron-15-3530:~/Desktop/Secure-Chat-Application/phase4$ g++ server.cpp -lcrypto
varshitha@varshitha-Inspiron-15-3530:~/Desktop/Secure-Chat-Application/phase4$ ./a.out
Server bound to port 5000
Server listening...
New client connected: 192.168.1.103
[PKI] Sending server certificate to 192.168.1.103
[DH] Shared key established with 192.168.1.103 | fingerprint: 10ae0ae7f789860f58a3e3f0bee51635e6f75ec9ba687812a2e75c9207f73817
User registered: sreeja -> 192.168.1.103
New client connected: 192.168.1.104
[PKI] Sending server certificate to 192.168.1.104
[DH] Shared key established with 192.168.1.104 | fingerprint: 2784a0c012a3b026038837cdaf76e9bd74918228053a8007ccdb6f07e22cdf
User registered: vaka -> 192.168.1.104
vaka -> sreeja: __E2E_INIT__0A068829444FCAD8DFC51A5D92641A9E971C486E8D80F0D8BA528402137330649472AB1EC80A01FAB311DC29DB88FAB6FF018DE5A8291E6845CDE9506ED836F7B4AF94F36C951D012452909507
D39C2D365CC25C768E8737E971E3577A3B515F6988899FE8A0B944551A18FEF2FB708DF56A29856E3B19DD8515DEB5061C70491FC65122EE6A75328C2DBD144B5FA6A39E4F6E6C8C27104DF00F3F9E70B64090B036EA069FE30CB277
0B18BDACB4F3CF58E8FC476AE2D859767D32534AD5C820C74D1A59D013647CDD144C840CFD469F94433DC7969098F2259C0EE665549D000CF6A9C0F79CD8A75F749323584999315340356F2C96EA904E5BD56C743F7B
sreeja -> vaka: __E2E_ACK__FC6867F119D5CD93A18C96CDCAE485A3683F0EF1A8C28B6F84CA4D87550642A16AF0E0DC699333C3F784A92EA37EABFA3344CD58F2E416C71F488C21734F0648044DE8DF11D8DA317883EA0C0ABA2
663618CEC286169FC137868887CB8506216C1FF4996E3CC08CC61D9059D2144B6D48E63A54104CC0F8C6B9A3B7336C738D688EA5D12E85A22955A31B9B8991A1E81E5812BFFD43E5F841AA8E7998111F222F8B60885C9FA5960FE2BD
6D9660C0CF257701B0283D0EA8464CA7281C6008700541C527DC90E8819F59097A690101743C0AAA360C89C79308439563D6AA5884E2D8B57485748C6B96769893D4A6287860DDE677CDFE6F2AA3894D879B32A0D13
vaka -> sreeja: __E2E_MSG__733a67a25364b21e109ce98d8909363a02c82e4f14f0e8c5fa7b9e85cac8
sreeja -> vaka: __E2E_MSG__3bbf42ef11872917fe1fda342993b79b9ae7f2c05ef8e38cb2e8d7dd0304f6819d
  
```

Figure 22: Server

```

sreeja@sreejalinux: ~/Desktop/phase4
Enter server port : 5000
[PKI] Waiting for server certificate...
[PKI] Validating certificate...
[PKI] Certificate is valid.
[PKI] Trusted CA: YES
[PKI] Validity period: OK
[PKI] Server identity: OK
[PKI] Verifying server private-key proof...
[PKI] Server proof-of-possession: VALID
[DH] Shared key established with server | fingerprint: 2784a0c012a3b026038837cdafc76e9bd749
18228053a8007ccdb6f070e22cdf

Connected to server 192.168.1.107:5000
All further traffic on this link is AES-256-GCM encrypted.

>
Hello from server. Please register.
> #register vaka
>
Successfully registered as vaka
> /chat sreeja
> @sreeja hi
>
You received this from client sreeja: hello
> /who
>
Online users:
- vaka
> /quit
Client exited.
sreeja@sreejalinux: ~/Desktop/phase4$

```

Figure 23: Client1 sending messages

```

sreeja@sreejalinux: ~/Desktop/phase4
sreeja@sreejalinux:~/Desktop/phase4$ g++ client.cpp -o client -lcrypto
sreeja@sreejalinux:~/Desktop/phase4$ ./client
Enter server IP: 192.168.1.107
Enter server port : 5000
[PKI] Waiting for server certificate...
[PKI] Validating certificate...
[PKI] Certificate is valid.
[PKI] Trusted CA: YES
[PKI] Validity period: OK
[PKI] Server identity: OK
[PKI] Verifying server private-key proof...
[PKI] Server proof-of-possession: VALID
[DH] Shared key established with server | fingerprint: 10ae0ae7f789860f58a3e3f0bee51635e6f75
ec9ba687812a2e75c9207f73817

Connected to server 192.168.1.107:5000
All further traffic on this link is AES-256-GCM encrypted.

>
Hello from server. Please register.
> #register sreeja
>
Successfully registered as sreeja
>
You received this from client vaka: hi
> /chat vaka
> hello
> /quit
Client exited.
sreeja@sreejalinux: ~/Desktop/phase4$

```

Figure 24: Client2 receiving messages

```

sreeja@sreejalinux: ~/Desktop/phase4
[PKI] Server identity: OK
[PKI] Verifying server private-key proof...
[PKI] Server proof-of-possession: VALID
[DH] Shared key established with server | fingerprint: 14035cc85d6748048e28bc6c
b15a00007f1caaf24af36d0000

Connected to server 192.168.1.107:5000
All further traffic on this link is AES-256-GCM encrypted.

>
Hello from server. Please register.
> #register sreeja
>
Successfully registered as sreeja
> @vaka hi
> [E2E] Key with client vaka | fingerprint: dfd827aa1d7b15085620319ad741678cd4d71207b2a5f
1d4c4b3372f6c4b5 | timestamp: 178853359
You received this from client vaka: hello
> [E2E] Key with client vaka | fingerprint: dc1a2d63cca872f4d396bee2703c2b9ad1e880e
5516df70e6c2f6e | timestamp: 178853375
>
You received this from client vaka: whatever
> /chat vaka
> [E2E] Handshake requested with client vaka.
> [E2E] Key with client vaka | fingerprint: 52d7adce03a9bac5363b57417dffec272d90003bce5a461
743160991c0b10432 | timestamp: 178853401
You received this from client vaka: hooray

```

Figure 25: Client sending e2e message

5 Phase-05

In this phase, we additionally implement a key expiry check whenever a client tries to send a message to another client. If the E2E key has expired, i.e., if the time elapsed since the key timestamp is ≥ 60 seconds, the client initiates a new E2E key exchange and uses the newly established key for communication. To achieve this, we store the timestamp associated with the E2E key, which is set to the time at which the E2E INIT is initiated. We also handle the case where both clients simultaneously initiate an E2E key exchange. To avoid ambiguity and ensure that both clients establish the same key, the E2E INIT with the earliest timestamp is given priority. If both E2E INIT messages have the same timestamp, the clients use the public key as a secondary tie-breaker to deterministically decide which exchange should be retained.

```
varshlth@varshlth:~$ ssh -i /root/.ssh/id_rsa -p 5000 root@192.168.1.103
Server bound to port 5000
Server listening...
New client connected: 192.168.1.104
[PKI] Sending server certificate to 192.168.1.104
[DH] Shared key established with 192.168.1.104 | fingerprint: fbc9ba5afdb37c9ac79ca8fd16bfdbace6bca16fff543dbc28bc754a9bfe239
User registered: vaka -> 192.168.1.104
New client connected: 192.168.1.103
[PKI] Sending server certificate to 192.168.1.103
[DH] Shared key established with 192.168.1.103 | fingerprint: d4202550ad0085f4692951f6f73879fb8b3df70e3f56ddc7e34e1c6fd71c15
sreeja -> vaka: _E2E_INIT_ 1788535622|A8A8266C5EA1D896F3528435431C6736A8805F8FA301D88B1613AD23B6A87080AFCDCFCFCD147B1FE47362159EBB1130ED03908C86E2656F6763A134B1B287F34EA25AC34FF
75D0A65692921A7F8992C02F166210F38C2D5AE5E939083C452637CD4272F46C3CA255C1C6F0A8921FAC70E46080CF9289022E799028166638074F1C63386FC8C293F024659101D8AF077383689F08452AF8011195084036F8
809D158B3714B4A0F0111452657502AD2D6C05115AC474BE1B1C84C5F9E84B318C926D05D40E67549C14CF62258E1E50D4565C0A89A30C2602FAB3F32E3DC4EAA1027CDDF82B7634F08047F0714150B44210
vaka -> sreeja: _E2E_ACK_ 1788535622|6A2A6C1024080212C70F857A71CF1390FF21FCF92F6A3CF989132BCBA7EA7A237F20676D619387CA270FC8C89386D1FEA89C406B116737A5B28BA354BA63219B3D1FA9971B1B77F4
99670DF0DFC50A8B86F4894D2A5E87FE1431245C2CA90ED266D10E833260FE10ED011A4C622D093653D0D993F3F0BC710DD0A64996DF8FCC5A6B8C75E8331ECBEA7EAC5946A4C2AE8FD94EADDBCECB8C750855E65C90A85
65E92D0D055776E390801B283F159240749FEA418797AEF1828F80A2C80CA14024B558E1328CA809954C54CEED2B9C3A8C6A80722CE14AF596B22708769A13048D4052CB2F87C86448FE4FEAE2A8C20A1CB8CA2
sreeja -> vaka: _E2E_MSG_ 2267717b3436984A385b78c408bce0d11356957b8d9b3f81d277b984Cf1a
vaka -> sreeja: _E2E_MSG_ fd08cf7ec6aa33a45d83e196e1204e55db16878a63d75347b4b4357fe2597adb
sreeja -> vaka: _E2E_MSG_ 111c7db150019d2789ab39a6f80ee4123367cf871a8f348fdaa15780d1ebdd
vaka -> sreeja: _E2E_MSG_ 3ed1ae7918b5e8f15ce75bd708df27b08b5accfb0d3384a5960748194ce7ba9d22
sreeja -> vaka: _E2E_MSG_ 546c99e9d26893808b3d737be47ba5a4d74e9edf22bbbaaf0bcd4acc0eb3
vaka -> sreeja: _E2E_INIT_ 1788535687|947C51040802337993896513080AC6A371491666650A332AF3309438637E38E79E083E79F985820896DF517C7606932C7084911139250586AF0F099C9E1E5EDD7F5AE3598518
5E5C8A8D5728205419A58AF898AE74C442739E91E36F33591C2AD3981558C3158A74231E8C096181AD09F59E2B67471798F06F76F365989880A137614804091C14AC3AD3110CF2C92A92459A2184D4F8C9F11441BD23527C782
DEF3196708A89D13F8D4FD684A284A5D5F3F2A8A9FAB91F82924521F8D852F58B7684CAF458C0940D6F893A7C38855108CC80375E4D6A8B93E4CCA708B81B5C408A9B17A89F1A4CC7E51F54834D1703FC814A16591A708C158C4
sreeja -> vaka: _E2E_ACK_ 1788535687|A48F354E017E999750D4438684d77373E55A9F54FCF50840D75F13286764E9D708F58E76609A3D9AC527755833594D04D37C88895DAE49D8DB85A8E98C98851F2659A808A7
E700EA10E5AD392CA68E894D88F8A299E34062CF17C3084051276554C04270E4498598B0C6C5394D6175334816434226FFC1788E0A0F14A40A95ED74F4180641B6888031403C57AD03E3CB95248D5D358C94908836099C98
6F877249244F3C419C43ED18C8F41859575B702BA3803718A6E6442448797991D58A59F7D01082D73CAEA8A3F9826C802311995D0677301F610DCD21A6C3C834B81C3001EC17347F2B387F37408934808EDF297D6CAE3
vaka -> sreeja: _E2E_MSG_ 808c1fb9eb493c36ee7f68fde47e1b948A71f6f46448c7233baaf86d32c50
sreeja -> vaka: _E2E_MSG_ 01db20661b6af494c1974532abc41b5d667da0669215276e032e5d98a2f9a
vaka -> sreeja: _E2E_INIT_ 1788535709|F280E8D0D6F961E883C45034C30F78178D76773E81D96E10A076824BF276D200A41028FC8D65781E18F9248F2A0B433D0AF64093BF26A28871093892CD0469C2B8C691081E081FC
4A8782C897A350ACF7152538B8FE172285E5CE0A8A42F04E80C8113C9E161722C9C8C02564E0A88C383E87777CAD9579F7482AEF58033440305F8643A20F72D14B03A5736F2D6F3AB930653624FD86D14C193CB2A8022CA453E89
7F80C7A8A8C322CD99668379886473D73CD208CD0A8E9E9423D8F498681786A144DAE79D9CA54980CF583995228980726956F774D9365C416812A802A33E206A99F7032872993C3A0338B8821539D3FE8713463607759522
sreeja -> vaka: _E2E_ACK_ 1788535709|CAEB7160F9F8B20B1C44D01D40B198E470E8508F33F80B7687A70582ED118491508189202742A6E5895F50F000E14307F41AC926A4A560F89208126672C86338927474A6D1E
1A72E27309180B2846975E0631AA252ADEDC3859A30B57D047443A8B950139AEAF326485C1AB8E5191ACF360708697AB10E683CF503A77C6952E4C5A52BA26F1303163A9996C63580E812EF10002F2595F3CDB8E925615
B07418E8F727263C94283D9A1962995439B2D3DAE2382ABAS18304C5EAE4389C562E1FAC68597C294C38C70E45C930C3787548258E73B5D0E6C0D678FC43905CCE71208BFD5229019D7EAA7058AA553095E8954F90DCA
sreeja -> vaka: _E2E_MSG_ 4b40d5830a5f593b17963d9a4355b5c87c0a0df4140d00b9e82498612b9f6d9682
vaka -> sreeja: _E2E_MSG_ 2505130587034474bf821c30cb0672b1a084b6696d817df426b6652236f6816591176353F9045de
```

Figure 26: Server

```
client [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Sep 4 13:29
sreeja@sreeja:linux:~/Desktop/phases
>
> Hello from server. Please register.
> #register sreeja
>
> Successfully registered as sreeja
> @vaka hi
> [E2E] Key with client vaka | fingerprint: 90afc40fab8726c0f3fcd6d419aa51fa753c
14434f25eb8f9d0ed1 | timestamp: 1788535622
>
> You received this from client sreeja: hi
>
> Error: chat partner not selected. Use /chat <username> First.
> /chat sreeja
> hello
>
> You received this from client sreeja: what
> > where
>
> You received this from client sreeja: who
> > blah
> [E2E] Key with client sreeja | fingerprint: 7816ac7fc04269e90ed2028e1de10e20a8f
21522d39654265bef4 | timestamp: 1788535687
>
> You received this from client sreeja: boo
> > /e2e sreeja
> [E2E] Handshake requested with client sreeja.
> [E2E] Key with client sreeja | fingerprint: cc4d092c9ffc4691adbe084a948efb8ee6c5c0d65b5
7bfff004ad1381610cfa | timestamp: 1788535709
>
> You received this from client sreeja: hooray
> > its working:)
```

Figure 27: Client sending e2e message

6 Prompts used

- Give basic server and client code using tcp connection with sockets, and i want to connect with my friends laptop whose ip addr is smtg smtg, and port - 5000
- so we want to send g,p,gâmodp from server to client initially while client requests connection, and then client knows,g,p,gâmodp and then seds ĝb modp from that cal gâb modp and then print this shared secret
- what is aes-gcm how does it encrypt and decrypt explain in detail
- there is some problem with tcp connectiosn , they are getting resent again and again, this mallory should act exactly like server all the messages and all sent to victim client should be same , and the clinets messgaes to be sent to server, so first of all verify properly if all that is happening bcuz my client is unable to reach this mallory do u understand

- i want to implement dh, aesgcm between server-client so keep the core logic in server, client files and keep the dh related implementation in dh.utils and aesgcm related implementation in aes.utils and use RFC 3526 MODP prime group for diffie hellman and do not use any built-in DH/ECDH or TLS/SSL libraries or APIs
- What are the commands used to generate certificate for server and how to setup CA, If i am both server and CA,How does client verify, explain step by step, And explain how the client verifies the validity period,expected server from the server certificate it gets,which functionality does it use, explain in detail, and this should happen before the dh exchange
- i want to add one extra level of encryption in this client.cpp file like whenever a client wants to talk to another client it currently sends the message to server and server decrypts, encrypt it again and send that to other client instead what we want to do is we want a dh exchange to happen between both clients first whenever they want to send a message to each other like clients exchange their public keys through server and server can see these obviously now if client wants to send a message it should encrypt it twice once with shared key between client and second with key between it and server server decrypts once encrypts it and the other client decrypts twice to read the message, the important thing is client shouldn't explicitly mention in cli that it wants this e2e dh exchange i.e. dh between both clients it should automatically be triggered and the initial registration message shouldn't be affected at all like client first sends #register it's username right to server after that is done when client wants to communicate with someone this should automatically happen and to differentiate between dh exchange and nrm chat messages between clients we include some prefixes in message like __E2E_INIT__, __E2E_ACK__, __E2E_MSG__
- now lets make one small change i.e. after every 60s the client e2e keys expire so each key created we need to store a timestamp also and that timestamp is created by the client sending e2e init it sends the timestamp along with the public key details in e2e init message itself and the other client will use this info if that other client also sent a init message to same client before receiving the init message from that client use the one with earliest timestamp amongst those two to break ties and ensure both the clients have same key also print that key to screen like the e2e key with this client is this in client cli remaining all should work exactly the same
- if the client is already registered and types in #register smtg it is being considered as a message currently but instead send a reply to client saying u have already registered or smtg like that
- i am adding a new feature in client here that if a client types in /e2e another client's name e2e handshake should happen between these two, in this case after handshake is successfully done, print the key fingerprint and timestamp as well, by a handshake i mean an e2e init message and an e2e ack should be sent back the existing key need to be expired and new one should be used further